- A *Docker registry* secret is used to give Kubernetes credentials to access a private Docker container registry. You'd use this if your organization stores all images in a private registry rather than pushing them to a public registry like Docker Hub. Use `kubectl create secret docker-registry`.

For your own application-specific secrets you should use the generic secret type. For example, run the following command to create

Once you've defined a secret, you can make it available to your pods in one of two ways:

- As files mounted in the filesystem inside your pods. For example, if you mounted the secret defined in listing 11.10 under the path `/etc/secrets/db` then you would end up with two files inside your pod: `/etc/secrets/db/username` and `/etc/secrets/db/password`. Your application can then read these files to get the secret values. The contents of the files will be the raw secret values, not the base64-encoded ones stored in the YAML.
- As environment variables that are passed to your container processes when they first run. In Java you can then access these through the `System.getenv("DB_USERNAME")` method call.

  TIP File-based secrets should be preferred over environment variables. It's easy to read the environment of a running process using `kubectl describe pod`, and you can't use environment variables for binary data such as keys. File-based secrets are also